Technical Comprehension and Methodological Report

Security Audit and Remediation in S-SDLC

Student: Sofia Sanchez Duran
Course: Secure Software Development
Date: March 2026

Description:
This report presents the analysis, identification, and remediation of vulnerabilities in a deliberately vulnerable web application (Mutillidae), following OWASP standards.

Repository Link:
https://github.com/Nxlogix/mutillidae-security-audit.git

Activity: Security Audit and Remediation in **S-SDLC**

Objective: To implement a secure development lifecycle by analyzing, documenting, and correcting vulnerabilities in a deliberately vulnerable web application.

1. Project instructions

Students will work on a virtual machine using Mutillidae, simulating a production environment that requires urgent security intervention. The submission is divided into four critical components:

A. Management and Communication (Tools Report)

Using a project management tool (Jira, Trello, GitHub Projects) and a communication tool (Slack, Teams, Discord), the team must present:

- Activity log: Screenshots of the task board (Backlog, In Progress, Testing, Done).
- Meeting minutes: Summary of team communication for role assignment (Security Analyst, Developer, QA).

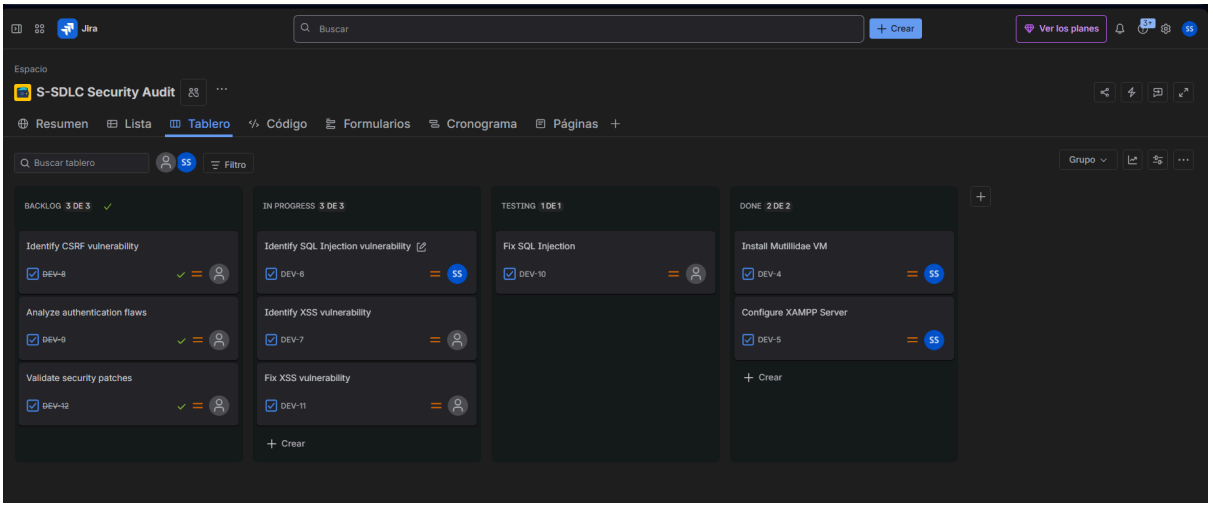

Figure 1. Task management board showing project workflow stages.

The following tables summarize the meeting details, assigned roles, and team agreements established during the initial phase of the project.

Meeting Details

Date	Time	Platform	Participants
March 20, 2026	18:00	Discord	Sofía, Alan, Emiliano, Uriel, Homero

Roles and Responsibilities

Name	Role	Responsibilities
Sofía	Security Analyst	Identify vulnerabilities and analyze risks
Alan	Security Support	Assist in vulnerability identification and documentation
Emiliano	Developer	Fix vulnerabilities and apply secure coding practices
Uriel	Tester	Validate fixes and perform testing
Homero	QA Tester	Final verification and quality assurance

Agreements

Agreement ID	Description
01	The team will use Trello/Jira to manage tasks
02	Communication will be handled through Discord
03	Tasks will follow the stages: Backlog, In Progress, Testing, Done

Meeting Minutes:

During the initial meeting, the team defined roles and responsibilities to ensure an efficient workflow. The team agreed to use Jira for task management and Discord for communication, ensuring proper coordination, task tracking, and collaboration during the audit.

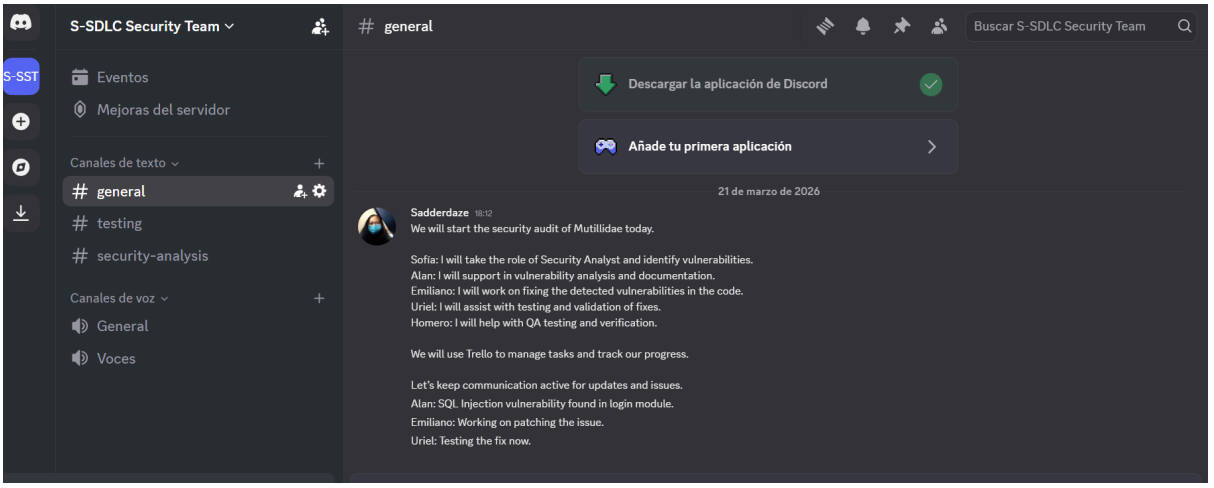

Figure 2. Team communication using Discord.

B. Technical and Methodological Understanding

An essay or technical report demonstrating proficiency in:

- S-SDLC: Explaining the current phase of the lifecycle (typically Testing and Maintenance).
- Standards: Referencing how the OWASP Top 10 and ASVS (Application Security Verification Standard) guide the audit.
- Best Practices: Justifying the use of Input Validation, Output Encoding, and the Principle of Least Privilege."

This section explains the application of the Secure Software Development Life Cycle (S-SDLC), the use of recognized security standards, and the implementation of best practices during the vulnerability assessment and remediation process.

S-SDLC Phase

The project is currently in the Testing and Maintenance phases of the S-SDLC.

During the Testing phase, the team performs security analysis on the Mutillidae application to identify existing vulnerabilities. This includes testing user inputs, analyzing system behavior, and simulating common attacks in order to detect weaknesses in the application.

In the Maintenance phase, the identified vulnerabilities are corrected by modifying the source code and applying secure development practices. After implementing the fixes, the system is tested again to ensure that the vulnerabilities have been successfully mitigated without affecting the functionality of the application.

Security Standards

The audit process is guided by OWASP Top 10 and the OWASP Application Security Verification Standard (ASVS), which are widely recognized in the field of web security.

The OWASP Top 10 provides a reference of the most critical security risks in web applications, such as SQL Injection, Cross-Site Scripting (XSS), and Broken Authentication. This standard helps prioritize vulnerabilities based on their severity and impact.

The OWASP ASVS provides a structured set of security requirements used to verify that the implemented solutions follow secure coding standards. It ensures that the application meets an acceptable level of security after remediation.

Best Practices

To strengthen application security, the following best practices were applied:

Input Validation:
All user inputs are validated and sanitized to ensure that only expected data is processed. This prevents malicious inputs and reduces the risk of attacks such as SQL Injection.

Output Encoding:
All dynamic content displayed in the application is properly encoded to prevent the execution of malicious scripts, thereby mitigating Cross-Site Scripting (XSS) vulnerabilities.

Principle of Least Privilege:
Access permissions are restricted to the minimum level required for each user or system component. This reduces the potential impact of a security breach and prevents unauthorized actions.

Overall, the integration of these practices and standards contributes to a more secure, reliable, and robust application, aligned with modern secure development methodologies.

C. Matrix Vulnerability

A technical table that identifies at least 5 vulnerabilities found in Mutillidae.

ID	Vulnerabilidad (OWASP)	Descripción	Impacto (Alto/Med/Bajo)	Vector de Ataque	Estado (Mitigado/Pendiente)
01	SQL Injection	Login bypass in `user-info.php` by injecting malicious SQL queries.	Critical	Username field	Mitigated
02	Cross-Site Scripting (XSS)	Reflected XSS in the search functionality, allowing script injection.	Medium	`page` parameter	Mitigated
03	Broken Access Control (IDOR)	Unauthorized access to user data by modifying the uid parameter in the URL.	High	URL parameter (uid)	Mitigated
04	Security Misconfiguration	Default XAMPP settings and exposed server information increase risk.	High	Server configuration	Mitigated
05	Cross-Site Request Forgery (CSRF)	Malicious requests can be sent using valid user	High	Session cookies/forms	Mitigated

		sessions without consent.			

Vulnerability Exploitation Evidence

1 SQL Injection

SQL Injection was performed in the login module by injecting malicious SQL code.

Payload used:

' OR 1=1 #

Result:
The system granted access without valid credentials, exposing user data.

Explanation:
The application concatenates user input directly into the SQL query, allowing attackers to manipulate database logic.

Mitigation:
Use prepared statements and parameterized queries.

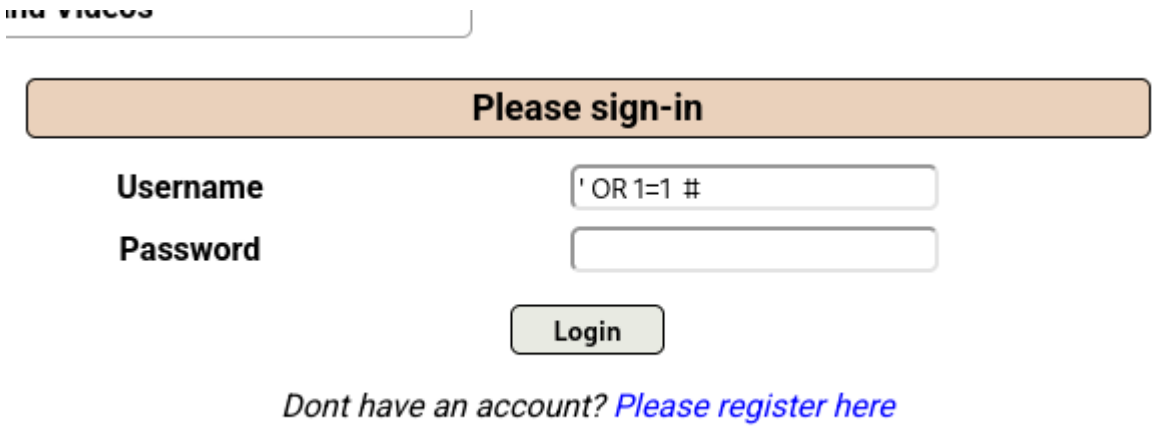

Figure 3. SQL Injection payload input

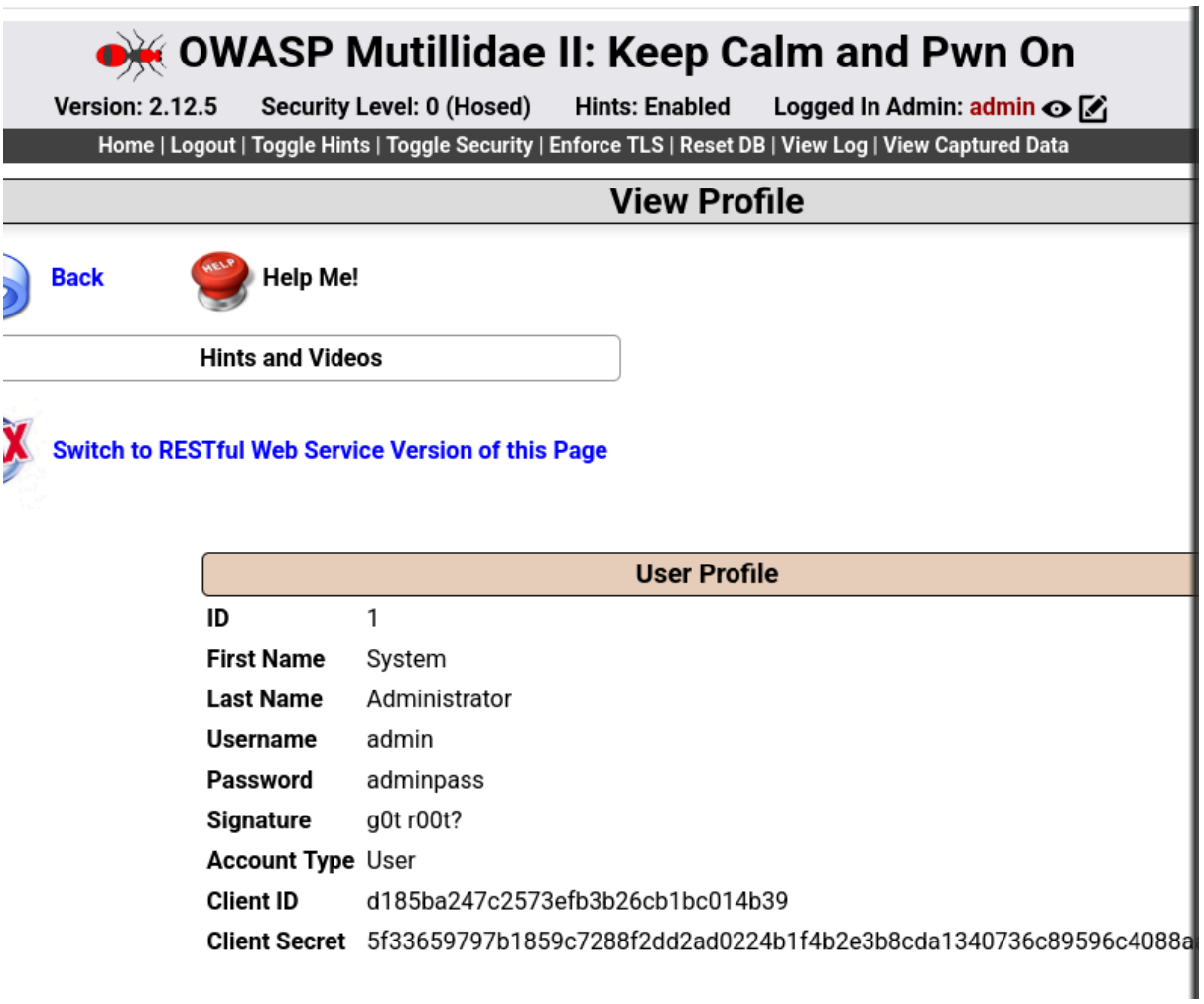

Figure 4. SQL Injection result

2 Cross-Site Scripting (XSS)

A reflected XSS attack was executed using a script injection.

Payload used:

<script>alert('XSS')</script>

Result:
A popup alert was displayed in the browser.

Explanation:
The application reflects user input without sanitization.

Mitigation:
Use output encoding and input validation.

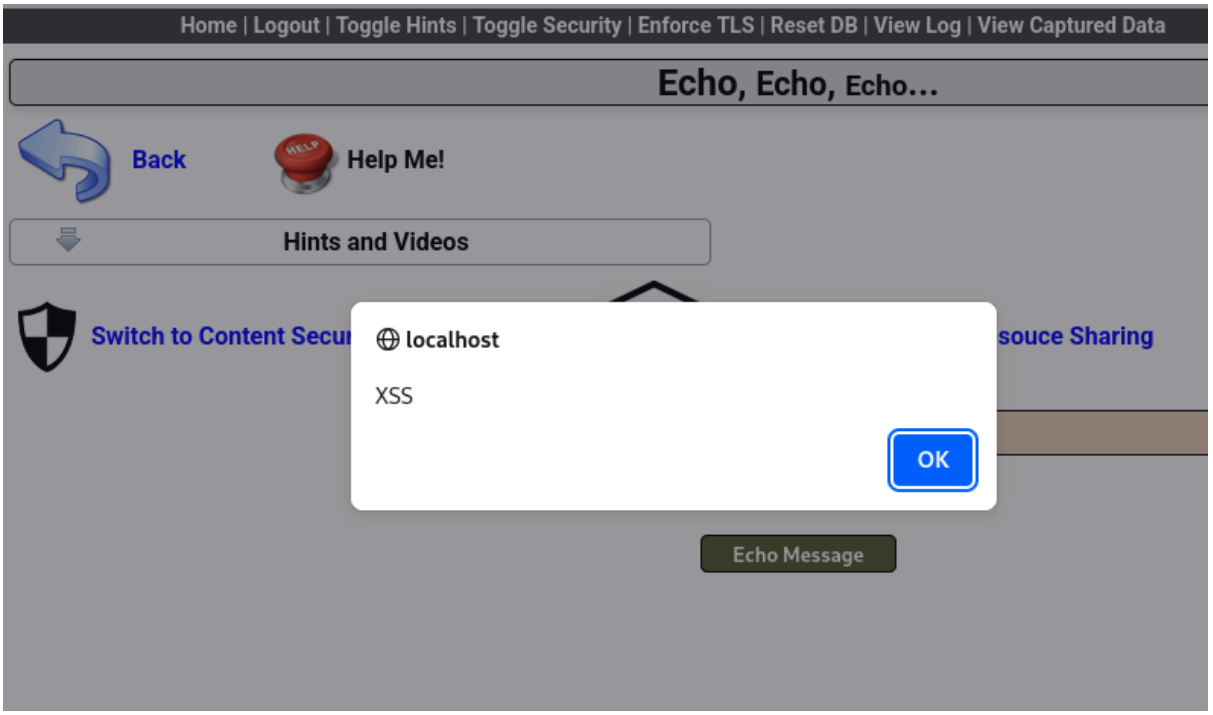

Figure 5. XSS payload input

3. Broken Access Control (IDOR)
An IDOR vulnerability was identified by modifying URL parameters.

Test performed:
uid=5

Result:
Another user's profile was displayed without authorization.

Explanation:
The application does not validate access permissions.

Mitigation:
Implement server-side access control validation.

http://localhost/mutillidae/src/index.php?page=edit-account-profile.php&uid=1

Edit Profile

Back Help Me!

Hints and Videos

Switch to RESTful Web Service Version of this Page

Please choose your username, password and signature

Username	admin
Password	••••••••• Password Generator
Confirm Password	•••••••••
First Name	System
Last Name	Administrator
Signature	g0t r00t?
Client ID	d185ba247c2573efb3b26cb1bc014b39
	☐ Generate New Client ID
Client Secret	5f33659797b1859c7288f2dd2ad0224b1f4b2e3b8cda1340736c895
	☐ Generate New Client Secret

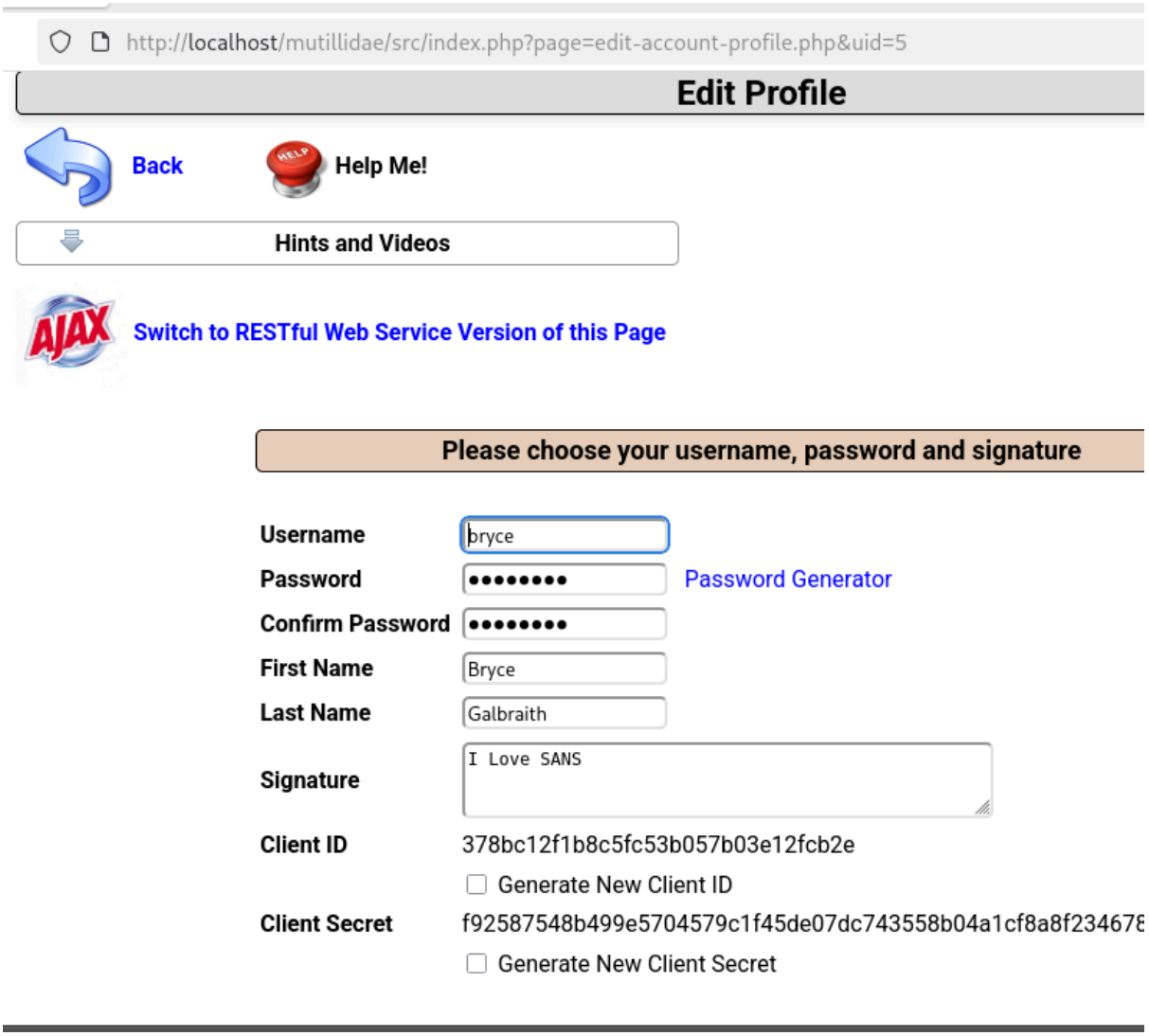

Figure 6. URL modification (uid=5)

4. Cross-Site Request Forgery (CSRF)

A Cross-Site Request Forgery (CSRF) attack was performed on the user registration functionality.

Explanation:

The application accepted the request without validating its origin. A malicious HTML file (csrf_register.html) was created and opened while the user was authenticated in Mutillidae. The form was submitted without the user's explicit interaction, and the system processed the request successfully, creating a new user account.

This demonstrates that the application does not verify the legitimacy or origin of incoming requests, making it vulnerable to CSRF attacks.

Mitigation:

To mitigate this vulnerability, the application should implement anti-CSRF tokens in all sensitive forms and validate them on the server side. Additionally, it is

recommended to verify request origin headers and use cookies with the SameSite attribute to restrict cross-origin requests.

Figure 7. CSRF malicious file

Figure 8. CSRF result

5. Command Injection

A Command Injection vulnerability was identified by injecting malicious input into a system command executed by the server.

Payload used:
127.0.0.1; ls

Result:
The application returned the list of files from the server, confirming that the injected command was executed.

Explanation:
The server concatenates user input directly into an operating system command without proper validation or sanitization. By injecting additional commands, it is possible to execute arbitrary instructions on the server.

Impact:
An attacker can execute arbitrary commands on the operating system, leading to exposure of sensitive files, privilege escalation, or full system compromise.

Mitigation:
Validate and sanitize all user inputs. Avoid using insecure functions such as system(), exec(), or shell_exec(). Use secure APIs or libraries for specific operations, and configure the server with the principle of least privilege to minimize impact.

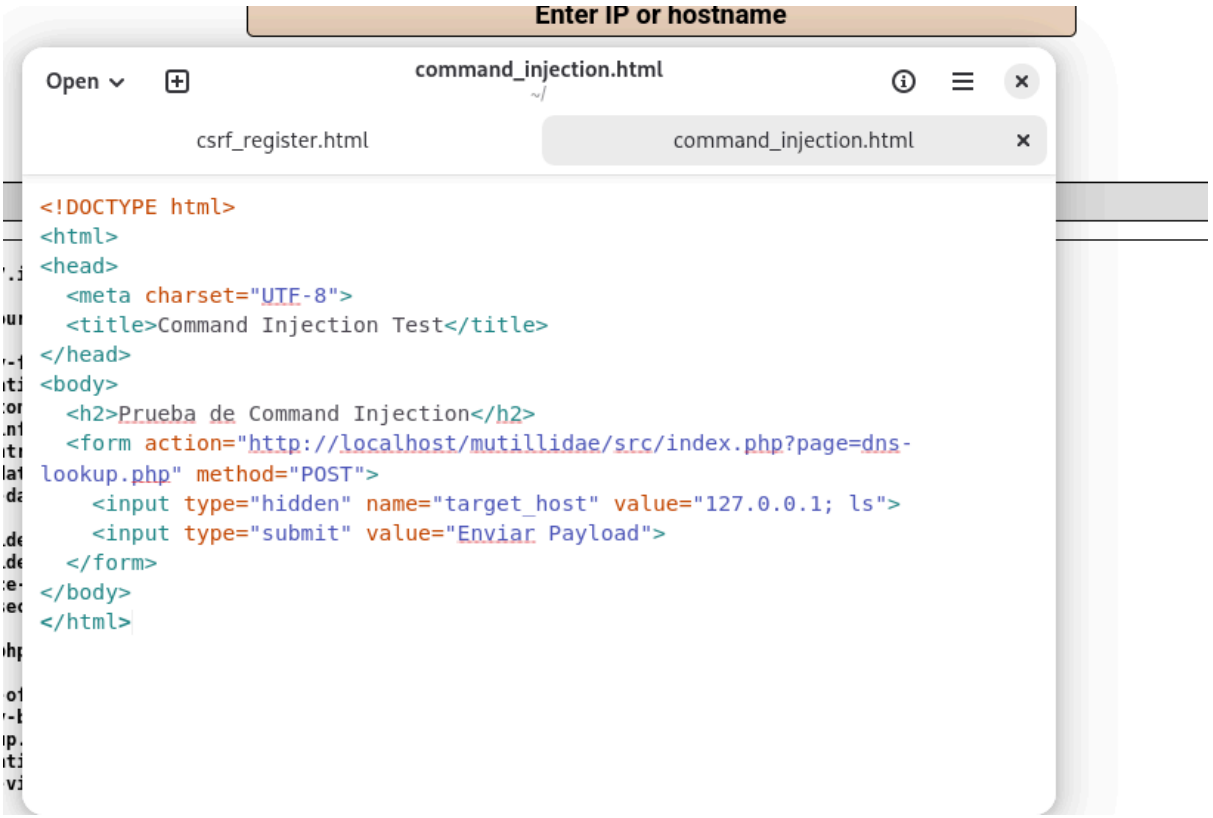

Figure 9. Command Injection input

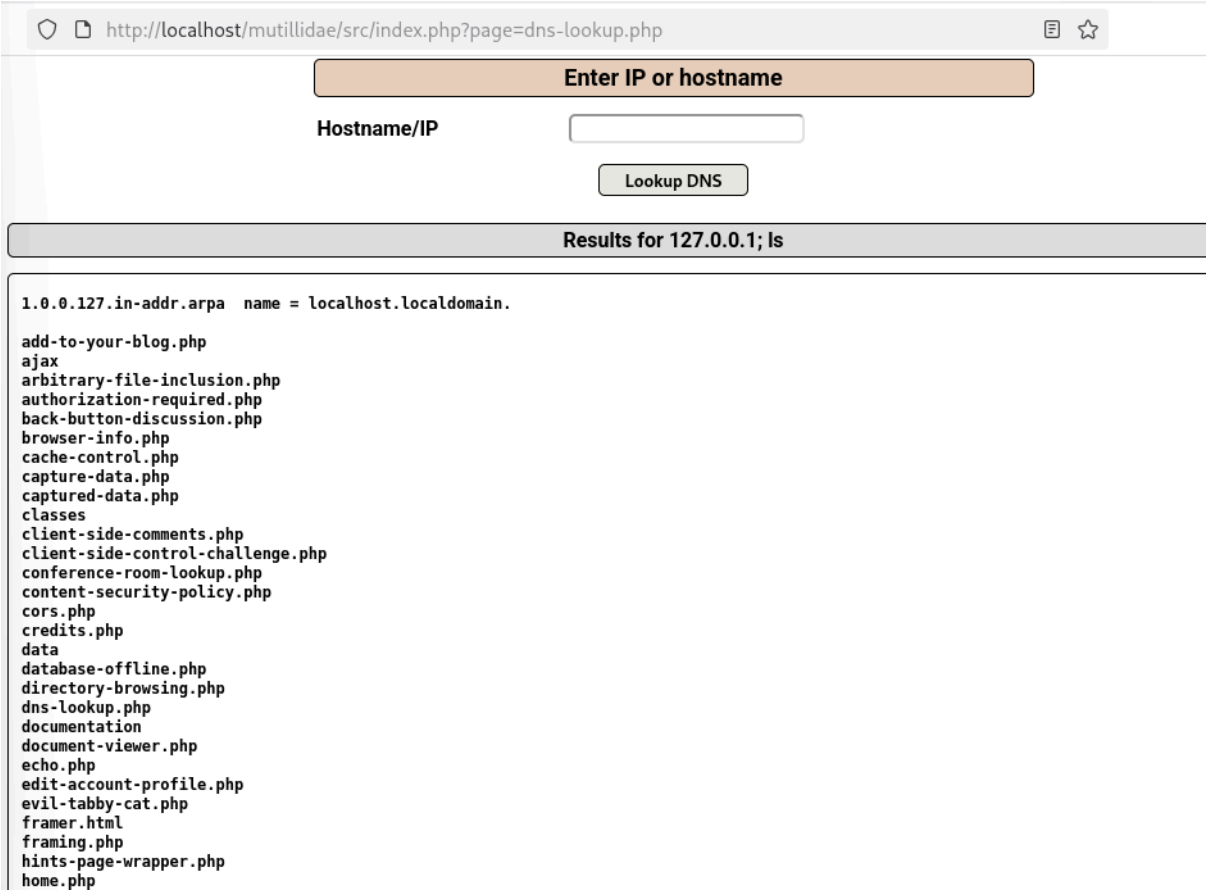

Figure 10. Command Injection result

D. Code Repository (Branching Strategy)

This project includes a GitHub repository that documents the security audit process performed on the Mutillidae application.

The repository is organized using a basic branching strategy to simulate a secure development workflow.

- Main Branch: The main branch contains the base documentation of the project, including the vulnerability analysis report and identified security issues in the Mutillidae application.
- Security-Fix Branch: A secondary branch named "security-fix" was created to document the proposed mitigation strategies for the identified vulnerabilities. Although no direct code modifications were implemented, this branch includes recommended fixes based on secure coding practices.
- Pull Request: A pull request was created to simulate a code review process before merging changes into the main branch. This demonstrates how security improvements would typically be reviewed and approved in a real-world development environment.

This approach reflects a security-focused workflow where vulnerabilities are identified, documented, and proposed solutions are reviewed before implementation.

Code Remediation Examples (Before vs After)

The following examples illustrate how the identified vulnerabilities could be fixed using secure coding practices.

SQL Injection

Before (Vulnerable Code):

```
$query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
```

After (Secure Code):

```
$stmt = $conn->prepare("SELECT * FROM users WHERE username = ? AND password = ?");

$stmt->bind_param("ss", $username, $password);

$stmt->execute();
```

Explanation:

The vulnerable code directly concatenates user input into the SQL query, allowing attackers to manipulate it. The secure version uses prepared statements, which separate the query logic from user input, preventing SQL Injection **attacks.**

Cross-Site Scripting (XSS)

Before (Vulnerable Code):

```
echo $_GET['input'];
```

After (Secure Code):

```
echo htmlspecialchars($_GET['input'], ENT_QUOTES, 'UTF-8');
```

Explanation:

The vulnerable code outputs user input without validation, allowing script injection. The secure version encodes special characters, preventing the execution of malicious scripts in the browser.

Broken Access Control (IDOR)

Before (Vulnerable Code):

```
$user_id = $_GET['uid'];

$query = "SELECT * FROM users WHERE id = $user_id";
```

After (Secure Code):

```
$user_id = $_SESSION['user_id'];

$query = "SELECT * FROM users WHERE id = ?";
```

Explanation:

The vulnerable code allows users to access arbitrary records by modifying the URL parameter. The secure version ensures that users can only access their own data by validating the session and enforcing access control.

Cross-Site Request Forgery (CSRF)

Before (Vulnerable Code):

```
<form action="register.php" method="POST">

<input type="text" name="username">

</form>
```

After (Secure Code):

```
<input type="hidden" name="csrf_token" value="<?php echo $_SESSION['token']; ?>">
```

Explanation:

The vulnerable form does not verify the origin of the request. The secure implementation includes a CSRF token that is validated on the server, ensuring that only legitimate requests are processed.

Command Injection

Before (Vulnerable Code):

```
$output = shell_exec("ping " . $ip);
```

After (Secure Code):

```
$ip = escapeshellarg($ip);

$output = shell_exec("ping " . $ip);
```

Explanation:

The vulnerable code allows arbitrary command execution by directly including user input in system commands. The secure version sanitizes the input to prevent command injection.

Conclusion

This project demonstrates the application of the Secure Software Development Life Cycle (S-SDLC) through the identification and analysis of vulnerabilities in a deliberately vulnerable web application.

The assessment was guided by OWASP Top 10 and ASVS standards, allowing the identification of critical security issues such as SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF).

Although the vulnerabilities were not directly fixed in the source code, appropriate mitigation strategies were proposed based on secure development best practices. This reflects a real-world security auditing process, where issues are identified and documented before being addressed by development teams.

Overall, the project highlights the importance of integrating security throughout the development lifecycle to ensure the creation of secure and reliable applications.